

Patrón de Diseño Singleton

1. Problema

En determinados sistemas es necesario garantizar que una clase tenga una única instancia durante toda la ejecución del programa. Esto ocurre cuando diferentes partes de una aplicación necesitan acceder al mismo objeto y compartir su estado.

Si se permitiera crear libremente varias instancias de esa clase, podrían generarse inconsistencias, duplicación de información o un uso innecesario de recursos.

Por ejemplo, una aplicación podría necesitar un único objeto encargado de administrar la configuración general del sistema. Si cada componente pudiera crear su propia instancia del administrador de configuración, cada instancia podría contener valores diferentes y provocar un comportamiento inconsistente.

Por lo tanto, el problema que busca resolver Singleton es controlar la creación de instancias de una clase y garantizar que exista solamente una instancia accesible desde diferentes partes del programa.

Situaciones en las que puede utilizarse

Algunos ejemplos de situaciones donde podría resultar útil son:

- Un administrador de configuración global.
- Un sistema centralizado de registro de eventos (logging).
- Un administrador de recursos compartidos.
- Un objeto que represente una configuración única de la aplicación.
- Un componente que deba ser compartido por diferentes partes del sistema.

2. Solución

El patrón Singleton propone que la propia clase sea responsable de controlar la creación de su única instancia.

Para conseguirlo, se utiliza una estructura compuesta principalmente por:

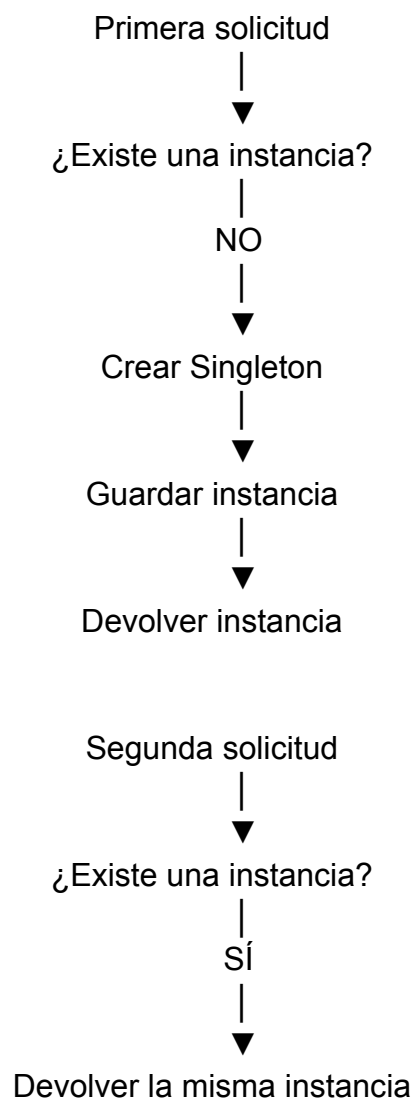
- Una clase Singleton.
- Un atributo privado que almacena la única instancia.
- Un constructor privado, que impide que otras clases creen objetos directamente.
- Un método público que permite obtener la instancia única.

En el diagrama UML desarrollado para este proyecto, estos elementos se representan mediante la clase Singleton, el atributo instancia, el constructor privado Singleton() y el método público obtenerInstancia().

El funcionamiento puede resumirse de la siguiente manera:

1. Una parte del programa solicita la instancia mediante obtenerInstancia().
2. Si todavía no existe una instancia, la clase crea una.
3. La instancia creada se almacena en el atributo instancia.
4. Las siguientes solicitudes devuelven la misma instancia.
5. De esta manera, no se crean nuevos objetos de Singleton.

Conceptualmente:



De esta forma, Singleton resuelve el problema restringiendo la creación de objetos y proporcionando un único punto de acceso a la instancia.

3. Consecuencias

La utilización del patrón Singleton presenta ventajas y desventajas que deben ser consideradas antes de incorporarlo a un sistema.

- Ventajas:

Garantiza una única instancia

La principal ventaja es que permite controlar la cantidad de objetos creados de una determinada clase, asegurando que exista solamente una instancia.

Proporciona un punto de acceso global

Las diferentes partes del programa pueden acceder a la instancia mediante un método común, sin necesidad de crear nuevos objetos.

Evita la creación innecesaria de objetos

Cuando una clase representa un recurso que debe ser compartido, reutilizar una única instancia puede evitar la creación de múltiples objetos equivalentes.

Centraliza determinados recursos

Puede ser útil cuando existe un recurso o servicio que conceptualmente debe ser administrado desde un único objeto.

- Desventajas:

Introduce un estado global

Al permitir el acceso global a una instancia, diferentes partes del programa pueden modificar su estado. Esto puede dificultar la comprensión del comportamiento del sistema.

Puede aumentar el acoplamiento

Las clases que utilizan directamente Singleton pasan a depender de una implementación concreta, lo que puede dificultar posteriormente reemplazarla por otra solución.

Puede dificultar las pruebas

Las pruebas unitarias pueden resultar más complejas cuando diferentes pruebas comparten el estado de una misma instancia.

Puede generar problemas de concurrencia

En aplicaciones con múltiples hilos, una implementación incorrecta de Singleton puede permitir que se creen varias instancias simultáneamente. Por este motivo, en sistemas concurrentes es necesario utilizar una implementación segura.

- Implicancias:

Singleton debe utilizarse con criterio. El hecho de que una clase necesite ser accesible desde diferentes partes del programa no significa necesariamente que deba convertirse en Singleton.

Su utilización es apropiada cuando realmente existe una necesidad de una única instancia controlada, especialmente cuando varios componentes deben compartir el mismo estado o recurso.

Sin embargo, utilizar Singleton de manera excesiva puede convertirlo en una dependencia global y dificultar el mantenimiento, la reutilización y las pruebas del software.

Por esta razón, antes de aplicar el patrón se debe analizar si la existencia de una única instancia es realmente un requisito del sistema.